



Domain Modeling & Development

Calendar	Round.3
Tag	BE_L2 presentation round3

루프팩 BE L2 - Round 3

| 객체 간 협력을 설계하고, 의도를 코드로 드러낸다.

Summary

- 도메인 모델링을 통해 현실 세계 개념을 코드로 표현하는 법을 배웁니다.
- Entity, VO, 도메인 서비스의 구분과 각각의 책임을 이해합니다.
- 유스케이스 기반으로 객체 간 협력을 설계하는 연습을 합니다.
- 레이어드 아키텍처와 DIP를 적용하여 테스트 가능한 구조를 구성합니다.
- 테스트 가능한 구조를 설계하고, 단위 테스트로 도메인 로직을 검증합니다.

Keywords

- 도메인 모델링
- Entity vs Value Object vs Domain Service
- 유스케이스 중심 설계
- 레이어드 아키텍처 + DIP
- 단위 테스트 기반 구현

Learning

Agent 와 의도한대로 협업하기

 이번주는 AI 를 더 다양하게 활용하는 방법에 대해 확장하지 않습니다.
대신, 이미 알고 있는 질문을 객체에 적용해봅니다.

| Rule-based Collaboration

- AI 를 단순히 질문을 생성하는 것이 아니라 **우리가 만든 규칙을 기준으로 판단하고, 행동**할 수 있도록 활용합니다.
- 본인이 생각하는 모델링 규칙, 아키텍처 등을 문서로 고정해 Agent 가 의도한 대로 동작할 수 있도록 합니다.
- 다음과 같은 질문들을 함께 고민하고, [CLAUDE.md](#) 를 업데이트 해, 더 잘 행동할 수 있도록 해 볼 예정입니다.

도메인 모델링

도메인 모델링은 **현실 세계의 개념과 규칙**을 소프트웨어 안에서 **객체 지향적으로 표현**하는 작업입니다.

- 단순한 클래스 설계가 아닌, **업무 지식을 담은 설계 행위**입니다.
- 핵심은 데이터가 아니라 **행위의 주체와 책임**입니다.
- 도메인 모델은 **해당 업무의 개념과 규칙**을 자연스럽게 반영할 수 있어야 합니다.

실무 사례로 알아보기

상품 좋아요 기능을 구현한다고 했을 때, 단순히 `상품.likeCount++` 로 설계했다가, 실제로는 **각 좋아요 하나가 고유한 의미를 가지고 누가 어떤 것을 눌렀는지, 언제 눌렀는지 등을 추적**해야할 필요로 인해 구조를 리팩토링하게 됩니다.

```
class Like(  
    val userId: Long,  
    val productId: Long,  
    val likedAt: LocalDateTime = LocalDateTime.now()  
)  
  
interface LikeRepository {  
    fun existsBy(userId: Long, productId: Long): Boolean  
    fun save(like: Like)  
    fun deleteBy(userId: Long, productId: Long)  
}  
  
class LikeService(  
    private val likeRepository: LikeRepository,  
    private val productRepository: ProductRepository  
) {  
    fun like(userId: Long, productId: Long) {  
        if (likeRepository.existsBy(userId, productId)) return  
        likeRepository.save(Like(userId, productId))  
        productRepository.incrementLikeCount(productId)  
    }  
  
    fun unlike(userId: Long, productId: Long) {  
        if (!likeRepository.existsBy(userId, productId)) return  
        likeRepository.deleteBy(userId, productId)  
        productRepository.decrementLikeCount(productId)  
    }  
}
```

▼ java

```
public class Like {  
    private final Long userId;  
    private final Long productId;  
    private final LocalDateTime likedAt;  
  
    public Like(Long userId, Long productId) {  
        this.userId = userId;  
        this.productId = productId;  
        this.likedAt = LocalDateTime.now();  
    }  
}  
  
public interface LikeRepository {
```

```

    boolean existsBy(Long userId, Long productId);
    void save(Like like);
    void deleteBy(Long userId, Long productId);
}

public class LikeService {
    private final LikeRepository likeRepository;
    private final ProductRepository productRepository;

    public LikeService(LikeRepository likeRepository, ProductRepository productRepository)
    {
        this.likeRepository = likeRepository;
        this.productRepository = productRepository;
    }

    public void like(Long userId, Long productId) {
        if (likeRepository.existsBy(userId, productId)) return;
        likeRepository.save(new Like(userId, productId));
        productRepository.incrementLikeCount(productId);
    }

    public void unlike(Long userId, Long productId) {
        if (!likeRepository.existsBy(userId, productId)) return;
        likeRepository.deleteBy(userId, productId);
        productRepository.decrementLikeCount(productId);
    }
}

```

👉 좋아요 개념을 독립된 도메인 개념으로 분리하여, **User ↔ Product** 관계를 추적하고 **유연한 확장성**을 확보합니다.

❌ 잘못된 구조 예시

```

class Product {
    val likedUserIds = mutableSetOf<Long>()
    fun like(userId: Long) {
        likedUserIds.add(userId)
    }
}

```

😞 문제 포인트 → 응집도가 낮고, 좋아요 자체가 가진 의미를 확장하기 어려움

✅ 개선 포인트 → 비즈니스 의미가 커질 수 있는 개념은 도메인 단위로 격리하는 것이 바람직합니다.

📦 Entity / Value Object / Domain Service

도메인 모델링의 핵심 구성 요소는 크게 세 가지로 나뉩니다.

각각의 개념은 책임과 성격이 다르며, **올바른 구분**이 유지보수성과 테스트 용이성에 큰 영향을 줍니다.

개념	책임과 성격	실전 예시
Entity	식별 가능한 고유 ID를 가지며, 상태 변화가 중요. 동일성은 ID로 판단. 시간이 지나 속성이 동일하더라도 연속성을 가짐	<code>User</code> , <code>Order</code> , <code>Product</code> 등 실제 상태가 존재하고 변경 가능한 객체
Value Object (VO)	“누구인지”가 중요하지 않고, “그 값이 무엇이나”만 중요한, 불변(immutable) 특성을 가지는 객체	<code>Money</code> , <code>Address</code> , <code>Quantity</code> 등 비교/계산 중심 객체 등 비교/계산 중심 객체
Domain Service	도메인 객체들이 직접 수행하기 어려운 도메인 로직을 위임받아 처리. 상태는 가지지 않음.	<code>PointChargingService</code> , <code>CouponApplyService</code> 등 다수 객체 협력이 필요한 로직

▼ QNA - Service 란 무엇일까요?

Service 는 도메인 계층에도 있고, Application 계층에도 있고, 심지어 infrastructure 계층에도 있습니다. 다만 어떤 계층에서 사용되느냐에 따라 부르는 이름이 달라집니다.

Service 의 대표적인 특징으로 2가지가 있습니다.

1. 상태를 갖지 않습니다. 즉, Input 와 output 이 명확합니다.
2. 서비스는 클라이언트에게 무엇가를 제공해주는 오브젝트나 모듈입니다.

▼ QNA - “행위자(doer)” 와 같은 Manager는 도메인인가요? 서비스인가? 사용해도 괜찮을까요?

- 어떤 로직이 있을 때, 이걸 마치 “도메인 객체인 것처럼” 만들어버리는 경우가 있는 것 같습니다
- 하지만 사실 이러한 객체는 그저 연산(예: 계산, 전송, 처리 등)만 수행하고, 고유한 상태나 정체성이 없습니다. doer, 즉 뭔가 “하는 놈”일 뿐, 그 자체로는 **도메인 개념**이 아닙니다.
- 이런 클래스는 자신의 고유한 상태(state)나 **도메인적 의미가 없고**, 단지 기능 하나 또는 연산 하나만 수행합니다. 확실한건, 비즈니스 개념을 풍부하게 담고 있지 않다는 것입니다.
- 그럼에도 불구하고, 도메인 모델을 더럽히지 않고, “로직을 따로 분리해서 명확하게 관리” 할 수 있다는 점에서는 유용합니다.

실무 사례로 알아보기

User(Entity) : 포인트 충전/차감이라는 '상태 변화'를 갖고, ID로 동일성을 판단합니다. 그리고 연속성을 가집니다.

```
class User(  
    val id: Long,  
    private var balance: Money  
) {  
    fun canAfford(amount: Money): Boolean = balance.isGreaterThanOrEqualTo(amount)  
  
    fun pay(amount: Money) {  
        require(canAfford(amount)) { "포인트가 부족합니다." }  
        balance -= amount  
    }  
  
    fun receive(amount: Money) {  
        balance += amount  
    }  
}
```

▼ java

```
public class User {  
    private Long id;  
    private Money balance;  
  
    public User(Long id, Money balance) {  
        this.id = id;  
        this.balance = balance;  
    }  
  
    public boolean canAfford(Money amount) {  
        return balance.isGreaterThanOrEqualTo(amount);  
    }  
  
    public void pay(Money amount) {  
        if (!canAfford(amount)) throw new IllegalArgumentException("포인트가 부족합니다.");  
        this.balance = this.balance.minus(amount);  
    }  
  
    public void receive(Money amount) {  
        this.balance = this.balance.plus(amount);  
    }  
}
```

```
}  
}
```

💡 User는 금액을 가지고 있는 상태를 표현하며, 포인트 사용과 충전이라는 명확한 행위를 책임지는 **도메인의 주체(Entity)**입니다. 도메인 규칙을 내부에서 스스로 수행하며, 책임과 상태를 함께 가진 구조를 통해 비즈니스 로직의 중심 역할을 수행합니다.

Money (VO) : 할인 가격 계산 시 사용되며, 값 자체가 동일하면 같은 객체로 간주합니다.

```
import java.math.BigDecimal  
  
@JvmInline  
value class Money(val amount: BigDecimal) {  
    init {  
        require(amount >= BigDecimal.ZERO) { "금액은 0 이상이어야 합니다." }  
    }  
  
    operator fun plus(other: Money): Money = Money(this.amount + other.amount)  
    operator fun minus(other: Money): Money = Money(this.amount - other.amount)  
    fun isGreaterThanOrEqual(other: Money): Boolean = this.amount >= other.amount  
}
```

▼ java

```
public record Money(BigDecimal amount) {  
    public Money {  
        if (amount.compareTo(BigDecimal.ZERO) < 0) {  
            throw new IllegalArgumentException("금액은 0 이상이어야 합니다.");  
        }  
    }  
  
    public Money plus(Money other) {  
        return new Money(this.amount.add(other.amount));  
    }  
  
    public Money minus(Money other) {  
        return new Money(this.amount.subtract(other.amount));  
    }  
  
    public boolean isGreaterThanOrEqual(Money other) {  
        return this.amount.compareTo(other.amount) >= 0;  
    }  
}
```

💡 Money는 단순 정수가 아니라, **도메인 규칙을 가진 값 객체**로 금액 연산의 안정성과 표현력을 높여줍니다. 이처럼 VO는 도메인의 무결성을 보장하고 테스트를 단순화하는 데 도움이 됩니다.

▼ 퀴즈! 만약 Address 의 구성요소가 시, 도, 군으로 구성되어 있습니다. 항상 VO 일까요?

= 어떤 Context 이냐에 따라 VO 가 될수도 안될 수도 있습니다.

우체국에서는 주소는 추적해야되는 대상이 될 수 있습니다. 그러므로 식별자를 가질 수도 있습니다.

PointChargingService : 유저 포인트 잔액 확인, 주문 금액 계산 등 여러 객체 협력이 필요한 로직은 별도의 도메인 서비스에서 처리합니다.

```
class PointChargingService {
    fun charge(user: User, amount: Money) {
        require(amount.amount > BigDecimal.ZERO) { "0원 이상만 충전할 수 있습니다." }
        user.receive(amount)
    }
}
```

▼ java

```
public class PointChargingService {
    public void charge(User user, Money amount) {
        if (amount.getAmount().compareTo(BigDecimal.ZERO) <= 0)
            throw new IllegalArgumentException("0원 이상만 충전할 수 있습니다.");
        user.receive(amount);
    }
}
```

💡 포인트 충전 로직은 단독 도메인으로 보기 애매하므로, Domain Service를 통해 유저 객체를 조작합니다.
이를 통해 **로직 재사용성과 테스트 용이성, 도메인 역할 분리**를 동시에 확보할 수 있습니다.

👥 Usecase 중심 객체 협력 설계

도메인 객체는 어떤 방식으로 책임을 나누면 좋을지 고민해 봅니다.
e.g. 유저가 상품을 여러 개 담아 한 번에 주문한다

📌 실무 사례로 알아보기 (주문 생성 Usecase)

흐름 예시

1. 유저가 주문 요청을 보냄
2. 주문 항목(Product + Quantity)을 기반으로 주문 생성
3. 각 상품의 재고 확인 및 차감
4. 유저의 포인트 잔액 확인 및 차감
5. 주문 정보를 저장

객체 역할 예시

객체	역할
Order	주문 항목 집합, 총 금액 계산, 생성 책임
Product	재고 확인 및 감소 책임
User	포인트 보유 및 차감 책임
OrderService	이들을 조합하여 유스케이스 단위로 처리

코드 예시

```
class OrderService(
    private val orderRepository: OrderRepository
) {
    fun createOrder(user: User, products: List<Pair<Product, Int>>): Order {
        val totalPrice = products.sumOf { (product, qty) -> product.price * qty }
    }
}
```



```

        if (!user.canAfford(totalPrice)) {
            throw IllegalStateException("잔액 부족")
        }

        products.forEach { (product, qty) -> product.decreaseStock(qty) }
        user.pay(totalPrice)

        val order = Order(
            user.id,
            products.map { (product, qty) -> OrderItem(product.id, qty) },
        )

        return orderRepository.save(order)
    }
}

```

▼ Java

```

public class OrderService {
    private final OrderRepository orderRepository;

    public OrderService(OrderRepository orderRepository) {
        this.orderRepository = orderRepository;
    }

    public Order createOrder(User user, List<Pair<Product, Integer>> products) {
        int totalPrice = products.stream()
            .mapToInt(p -> p.getFirst().getPrice() * p.getSecond())
            .sum();

        if (!user.canAfford(totalPrice)) {
            throw new IllegalStateException("잔액 부족");
        }

        for (Pair<Product, Integer> pair : products) {
            pair.getFirst().decreaseStock(pair.getSecond());
        }
        user.pay(totalPrice);

        List<OrderItem> items = products.stream()
            .map(p -> new OrderItem(p.getFirst().getId(), p.getSecond()))
            .collect(Collectors.toList());

        return orderRepository.save(new Order(user.getId(), items));
    }
}

```

💡 복잡한 협력은 반드시 테스트 가능한 구조로 끊어서 설계되어야 합니다.

유연한 소프트웨어 아키텍처

레이어드, hexagonal, 클린 등 많은 소프트웨어 아키텍처 이론들이 대두되고 있어요.
중요한 것은 어떤 아키텍처냐 보다 얼마나 유연하게 잘 구성하느냐 입니다.
우리는 레이어드 아키텍처에 DIP 를 적용해 지키기 쉬우면서, 변화에 용이한 아키텍처를 채택합니다.

Layered Architecture 와 책임 분리

```
[ Interfaces Layer ]
→ 사용자와 직접 연결되는 Web/Controller
→ package rule : /interfaces/api/xx

[ Application Layer ]
→ 유스케이스 실행, 흐름 조율 (Facade, Service)
→ package rule : /application/xx

[ Domain Layer ]
→ 비즈니스 로직의 핵심 (Entity, VO, DomainService, Repository )
→ package rule : /domain/xx

[ Infrastructure Layer ]
→ 외부 기술 의존 영역 (JPA, Redis, Kafka 등)
→ package rule : /infrastructure/xx
```

📌 실무 적용 포인트

각 계층은 **명확한 책임과 관심사**를 가집니다.
도메인은 스스로 책임지는 구조여야 테스트 가능성이 높아집니다.

- **Interfaces Layer**
 - Application Layer 의 유스케이스 호출 책임만을 갖습니다.
 - 필요에 의해 요청 객체에 대한 검증, 응답 객체 매핑 등을 수행할 수 있습니다.
- **Application Layer**
 - 각 비즈니스 기능의 **흐름**을 조율해 유스케이스 함수들을 완성하여 제공합니다.
 - 실질적인 비즈니스 로직들은 최대한 도메인으로 위임할 수 있도록 합니다.
- **Domain Layer**
 - **도메인 로직**들은 도메인 계층에 위치하며, 다른 계층에 의존하지 않도록 합니다.
 - 비즈니스의 중심이며, 모든 의존 방향은 도메인 계층을 향합니다.
- **Infrastructure Layer**
 - 도메인 객체, 데이터 등을 저장, 조회 하는 기능에 대한 구현을 제공합니다.
 - 구체적인 외부 기술에 의존하며 도메인 계층이 원하는 기능을 제공할 수 있어야 합니다.

🔄 DIP (Dependency Inversion Principle) 적용

의존성 방향을 **도메인** → **인프라**가 아닌, **도메인 (인터페이스)** ← **인프라 (구현체)** 로 뒤집을 거예요.

```
// Domain Layer
interface OrderRepository {
    fun save(order: Order): Order
}

// Infrastructure Layer
@Repository
class OrderRepositoryImpl(...) : OrderRepository {
    override fun save(order: Order): Order {...}
}
```

▼ Java

```
// Domain Layer
public interface OrderRepository {
    public Order save(Order order);
}
```



```
// Infrastructure Layer
@Repository
public class OrderRepositoryImpl(...) implements OrderRepository {
    @Override
    public Order save(Order order) {...}
}
```

📌 실무 적용 포인트

- 테스트 시엔 `FakeOrderRepository` 나 `InMemoryOrderRepository` 를 활용하여 독립적인 테스트 가능
- 구조적 유연성 확보 (DB 교체, 비즈니스 로직 변경에도 영향 최소화)
- 테스트 용이성 확보 (Mocking 가능)



References

구분	링크
도메인 주도 설계	위키 - 도메인 주도 설계
객체 지향 원칙	나무위키 - 객체 지향 원칙 (SOLID)
Layered Architecture	baeldung - 레이어드 아키텍처
Design Pattern	리팩토링 구루 - 디자인 패턴

객체 지향 원칙과 디자인 패턴은 개발자로서 성장할 수록 더욱 더 중요하게 느껴지는 항목입니다. 시간이 날 때마다 학습하면 많은 도움이 될 거예요.



Next Week Preview

이번 주에는 **객체 간 협력**과 **도메인 중심 설계**를 학습했습니다. 하지만 실제 애플리케이션에선 단순한 도메인 로직만으로는 끝나지 않습니다.

다음 주엔 아래와 같은 실전 상황을 마주합니다:

- 동시에 여러 사용자가 주문을 시도할 때 **재고가 음수가 되는 문제**
- 결제와 재고 차감, 포인트 사용을 동시에 다룰 때 **정합성 유지 문제**
- 트랜잭션이 실패했을 때 **어떤 기준으로 롤백할 것인지**

즉, 다음 라운드는 **도메인 객체를 엮어서, 유스케이스**를 완성하는 주차입니다. 그리고 이 과정에서 **트랜잭션, 동시성, 실패 복구 설계**를 고민하게 됩니다.